

Pretraining, fine-tuning oraz nauka reprezentacji

Spis treści



1. Pretraining oraz fine-tuning – omówienie
2. Pretraining oraz fine-tuning – przykład praktyczny
3. Leakage, overfitting
4. Wstęp do uczenia samonadzorowanego

Pretraining – zarys problemu

Wyobraźmy sobie następującą sytuację:

Jakiś czas temu trenowaliśmy dużą sieć konwolucyjną klasyfikującą obrazy różnych obiektów, takich jak koty, samochody itp.

Teraz stoi przed nami zadanie stworzenia modelu rozpoznającego choroby na zdjęciach medycznych.

Czy jest metoda na wykorzystanie wiedzy zdobytej podczas nauki poprzedniego problemu do przyśpieszenia treningu nowego modelu?

TAK! – odpowiedzią jest **pretraining!**

Pretraining – ogólny schemat

Posiadamy model
wytrenowany na
zbiorze danych **A**



Dostajemy nowy zbiór danych **B**
wraz z nowym zadaniem. Zamiast
uczyć na nim nowy model od zera,
wykorzystujemy parametry modelu
uczonego na **A**.



Douczamy nowy
model na zbiorze **B**.

Pretraining

Fine-tuning



Zalety pretrainingu

- Szybsza zbieżność
- Łatwiejsza nauka na małych zbiorach danych
- Zmniejszona ilość wymaganych etykiet
- Poprawa generalizacji
- Bardziej przewidywalny trening

Pretraining – główne kategorie

Pretraining zwykle dzielimy na dwie główne kategorie:

- **Pretraining nadzorowany**

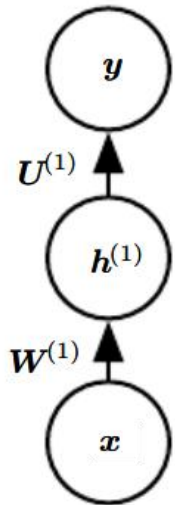
Model pretrenowany uczy się na danych z etykietami – rozwiązuje zadania predykcji/klasyfikacji. Wiedza zdobyta podczas treningu może być wykorzystana w pokrewnych problemach.

- **Pretraining nienadzorowany**

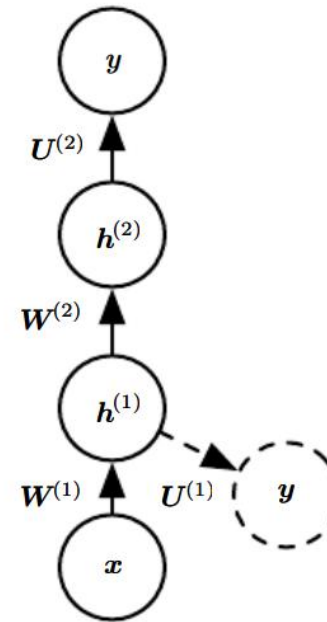
Model jest pretrenowany na zbiorach danych bez etykiet. Wykorzystuje pretekstowe zadania w celu odkrycia zależności w danych.

Pretraining nadzorowany - przykłady

Greedy supervised pretraining



Trenujemy płytką
architekturę na
konkretnym
zadaniu



Stopniowo
dodajemy kolejne
warstwy ukryte

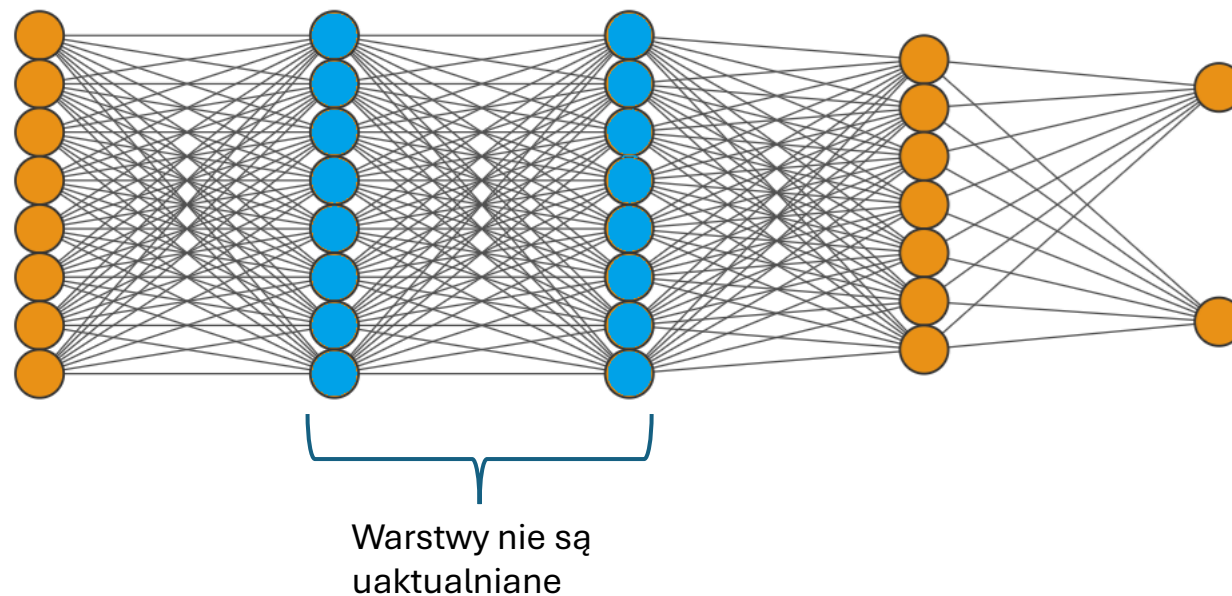
Zamrażanie warstw

Zamrażanie warstw w sieciach neuronowych jest metodą polegającą na wyborze niektórych warstw modelu, które nie będą uaktualniane w procesie uczenia.

Zalety:

- Zmniejszenie kosztów obliczeń
- Szybszy trening
- Potencjalne zwiększenie umiejętności generalizacji modelu

Ilość zamrożonych warstw powinna być proporcjonalna do stopnia podobieństwa problemu!



Pretraining nadzorowany – przykłady cd.

Transfer learning

Wykorzystanie wiedzy zgromadzonej w modelu trenowanym pewnym zadaniu do przyspieszenia nauki podobnego zadania.

ImageNet



<https://www.kaggle.com/datasets/dimensi0n/imagenet-256>


Fine-tuning

Cats vs dogs

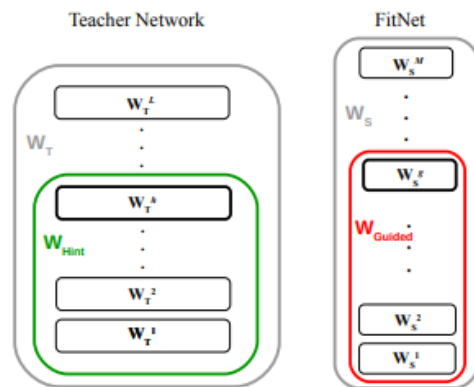


https://www.tensorflow.org/datasets/catalog/cats_vs_dogs

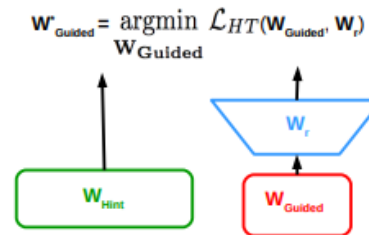
Pretraining nadzorowany – przykłady cd.

Knowledge distillation (FitNets)

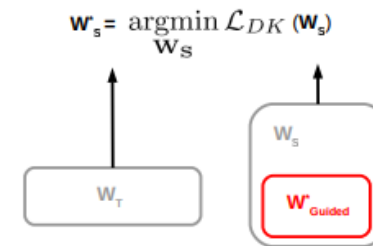
Mając do dyspozycji szeroką (posiadającą dużo neuronów w warstwach ukrytych), ale nie tak głęboką sieć neuronową **nauczyciela**, trenujemy za jej pomocą mniejszą sieć **studenta** (głębszą, ale posiadającą mniej neuronów w warstwach ukrytych).



(a) Teacher and Student Networks



(b) Hints Training



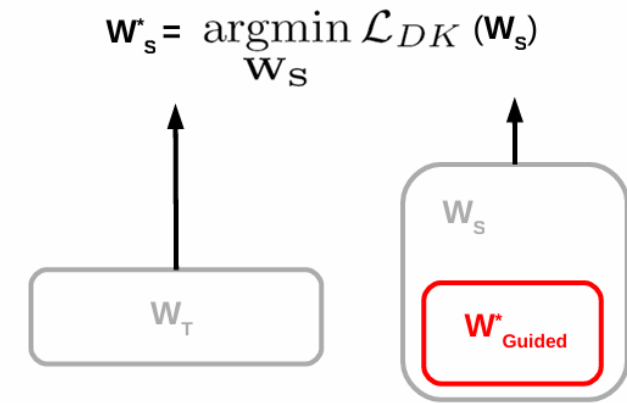
(c) Knowledge Distillation

Pretraining nadzorowany – przykłady cd.

Knowledge distillation (FitNets)

Głównym zadaniem sieci jest minimalizacja funkcji straty

$$\mathcal{L}_{KD}(\mathbf{W}_S) = \underbrace{\mathcal{H}(\mathbf{y}_{\text{true}}, P_S)}_{\text{Błąd klasyfikacji sieci studenta}} + \lambda \underbrace{\mathcal{H}(P_T^\tau, P_S^\tau)}_{\text{Różnica między aktywacjami ostatnich warstw sieci nauczyciela i studenta}}$$

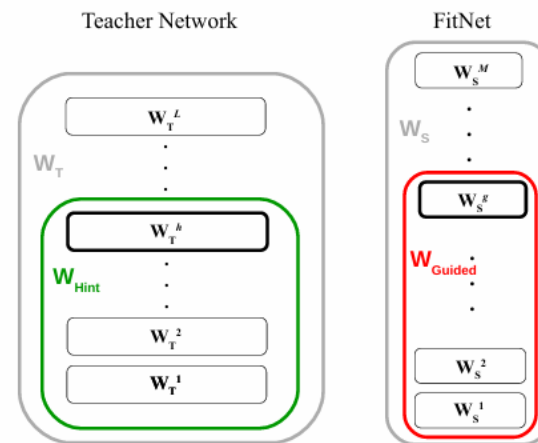


(c) Knowledge Distillation

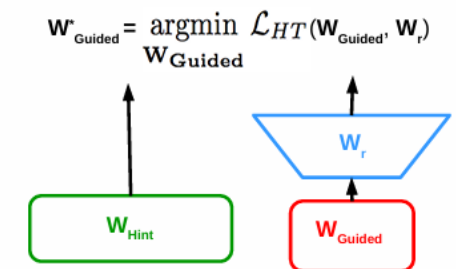
Pretraining nadzorowany – przykłady cd.

Knowledge distillation (FitNets)

Dodatkowo sieć nauczyciela daje dodatkowy sygnał sieci studenta – podaje jej aktywację wybranej warstwy ukrytej:



(a) Teacher and Student Networks



(b) Hints Training

$$\mathcal{L}_{HT}(\mathbf{W}_{\text{Guided}}, \mathbf{W}_{\mathbf{r}}) = \frac{1}{2} \left\| \underbrace{u_h(\mathbf{x}; \mathbf{W}_{\text{Hint}})}_{\text{Reprezentacja w warstwie ucznia}} - \underbrace{r(v_g(\mathbf{x}; \mathbf{W}_{\text{Guided}}))}_{\text{Reprezentacja w warstwie studenta}}; \mathbf{W}_{\mathbf{r}} \right\|^2,$$

Przerywnik praktyczny

Fine-tuning

Fine-tuning to proces polegający na adaptowaniu pretrenowanego modelu do wykonywania nowego zadania.

Adaptowany model zwykle był trenowany na dużych zbiorach danych zawierających dane ogólnego zastosowania (np. duże modele językowe trenowane na ogromnych korpusach nieoznaczonych danych).

Dostrajanie polega wtedy na dopasowaniu modelu pretrenowanego do węższej, bardziej szczegółowej dziedziny (np. poprawa rozumowania modelu językowego).

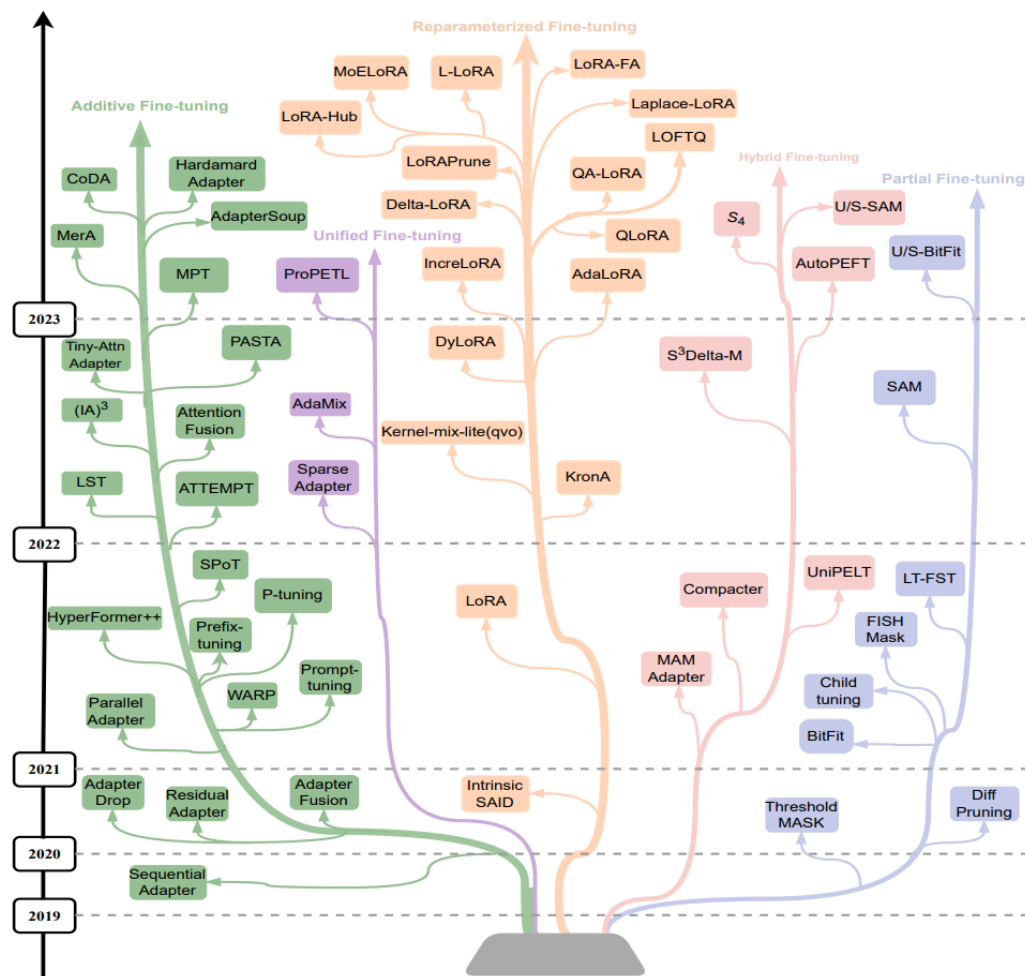
Typowo podczas fine-tuningu trening przebiega z mniejszą **stałą uczenia**, aby zmniejszyć ryzyko **katastroficznego zapomnienia**.

Fine-tuning - przykłady

Podstawowe metody dostrajania modeli już omówiliśmy:

- Pełen fine-tuning modyfikujący wszystkie wagi modelu
- Pozostawienie niektórych warstw bez zmian za pomocą zamrażania warstw

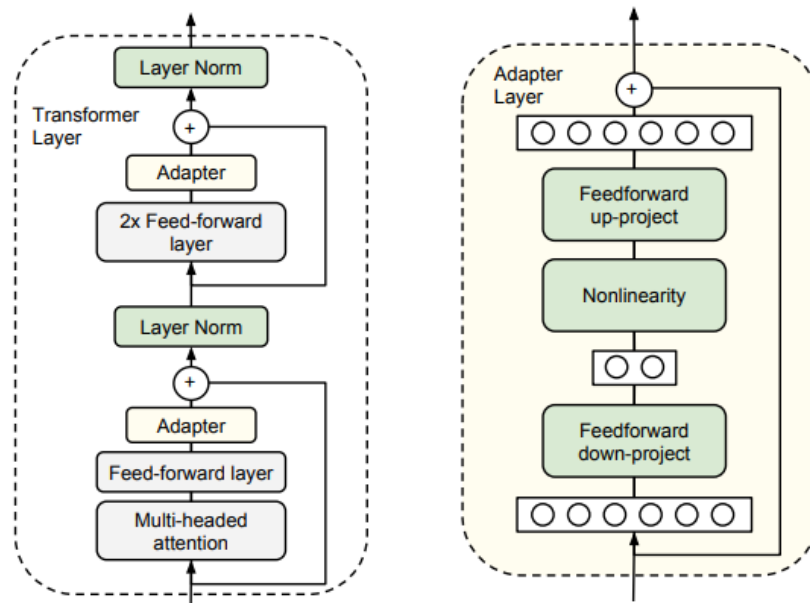
Fine-tuning - przykłady



Mnogość
nowoczesnych
technik dostrajania
dużych modeli

Fine-tuning - przykłady

Adapter tuning – dodawanie małej liczby nowych parametrów, trenowanych na nowym zadaniu, przykładowo dodawanie nowych warstw do istniejącego modelu. Oryginalne warstwy pozostają nienaruszone.

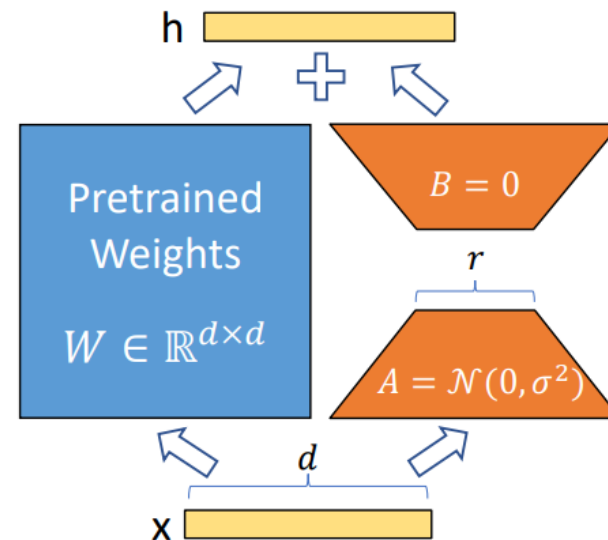


Po lewej – dodanie adaptera dwukrotnie w pojedynczej warstwie transformera

Po prawej – architektura adaptera

Fine-tuning – przykłady cd.

LoRA (low-rank-parametrized update matrices) - symulowanie pełnego treningu parametrów modelu za pomocą niskowymiarowych macierzy



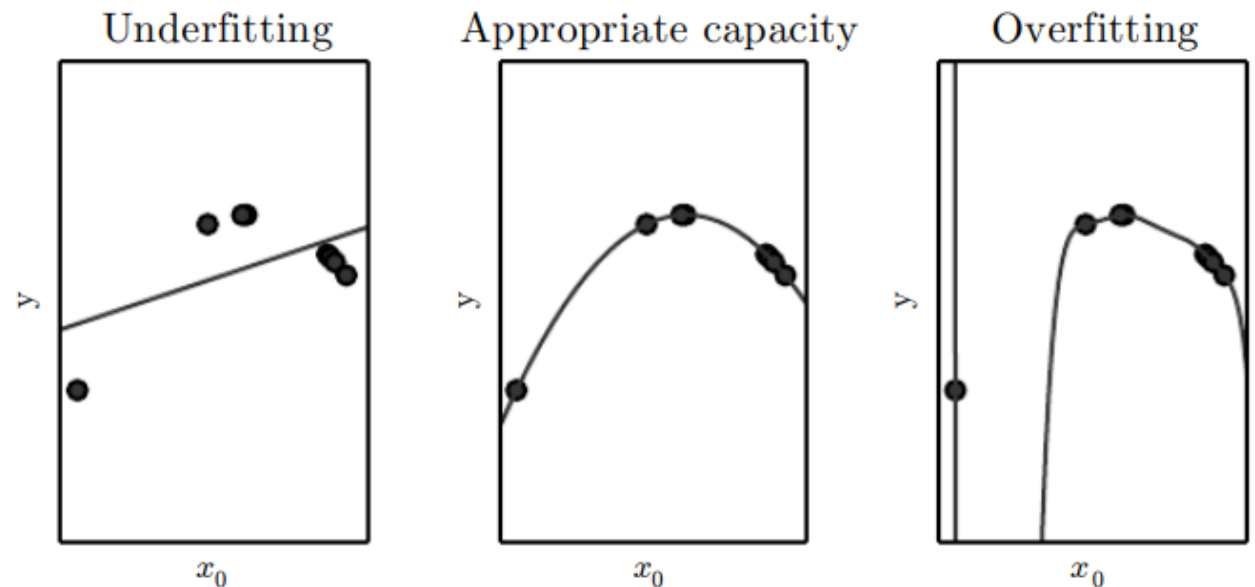
Oryginalne wagi W pozostają zachowane, a treningowi podlegają jedynie macierze A i B .
Zmiana macierzy wag ΔW jest przybliżana za pomocą iloczynu AB

Przerywnik dobropraktykowy

Overfitting

Overfitting (przeuczenie) to fenomen występujący w uczeniu maszynowym polegający na zbyt nim dopasowaniu modelu do danych treningowych. Rzeczywiste dane obarczone są szumem, do którego model o zbyt dużej **pojemności** może zacząć się dopasowywać.

Nieformalnie przez pojemność rozumiemy miarę mówiącą o zdolności modelu do dopasowania się do dużej ilości funkcji.



Overfitting

Overfitting podczas **fine-tuningu** może powodować **katastroficzne zapomnianie** – proces, w którym model traci nauczane w fazie pretreningu informacje, co drastycznie zmniejsza jego umiejętność generalizacji.

Zapobieganie przeuczeniu podczas dostrajania

- Zamrażanie warstw
- Zmniejszona/adaptacyjna stała uczenia
- Wykorzystanie nowoczesnych technik dostrajania
- Augmentacja danych

Wyciek danych

Wyciek danych to sytuacja, w której model otrzymuje informacje podczas treningu, do których nie miałby dostępu podczas w fazie testowania.

Pojawienie się wycieku danych powoduje między innymi zawyżone metryki jakości modelu oraz gorsze dopasowanie modelu.

Wyciek danych – standardowe przykłady

Do standardowych przykładów wycieku danych, znanych z klasycznego uczenia maszynowego, zaliczamy między innymi:

- Normalizacja danych przed podziałem treningowym,
- Obecność kolumn pośrednio informujących o zmiennej celu,
- Obecność danych testowych w zbiorze treningowym,
- Dzielenie zbioru danych o charakterze sekwencyjnym (temporalnym) na zbiór testowy i treningowy w sposób losowy.

Wyciek danych podczas dostrajania

Podczas dostrajania modelu należy być uważnym – dostrajane modele często były pretrenowane na dużych zbiorach danych. Jeżeli w ich skład wchodziły niektóre (bądź wszystkie) dane używane do fine-tuningu, wynikowy model może prezentować zawyżone metryki jakości podczas testowania!

Wstęp do nauki reprezentacji

Czym jest uczenie reprezentacji?

Representation learning (nauka reprezentacji) to jedno z zadań uczenia maszynowego, którego celem jest stworzenie nowych reprezentacji danych, które łatwiej następnie wykorzystać w innych zadaniach.

Jak uczymy się reprezentacji dziś?

Obecnie do nauki reprezentacji często wykorzystuje się **uczenie samonadzorowane** – paradygmat uczenia maszynowego, w którym modele same tworzą „etykiety” na nieoznaczonych danych.

Dlaczego uczenie samonadzorowane?

Ponieważ uczenie samonadzorowane nie wymaga etykiet, pozwala na tworzenie pretrenowanych modeli w oparciu o ogromne zbiory danych. Reprezentacje pochodzące z takich modeli są bardzo przydatne z różnych zadaniach nadzorowanych.

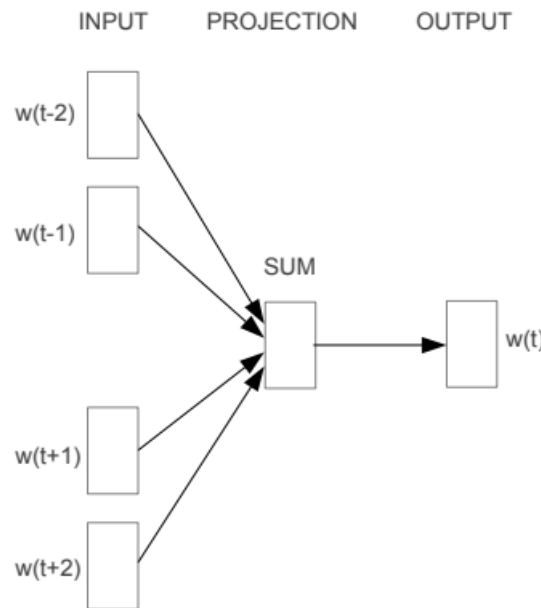
Przykłady zadań samonadzorowanych

- Przewidywanie brakujących słów w określonym kontekście
- Odtwarzanie zaszumionego obrazu
- Uczenie kontrastowe

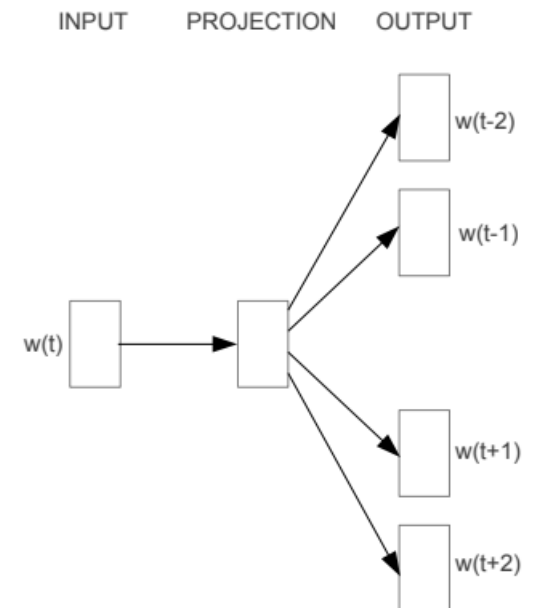
Modele samonadzorowane

Word2vec

Model uczący się przewidywać centralne słowo w zadanym kontekście. Alternatywnie przewidujemy kontekst na podstawie słowa centralnego.



CBOW

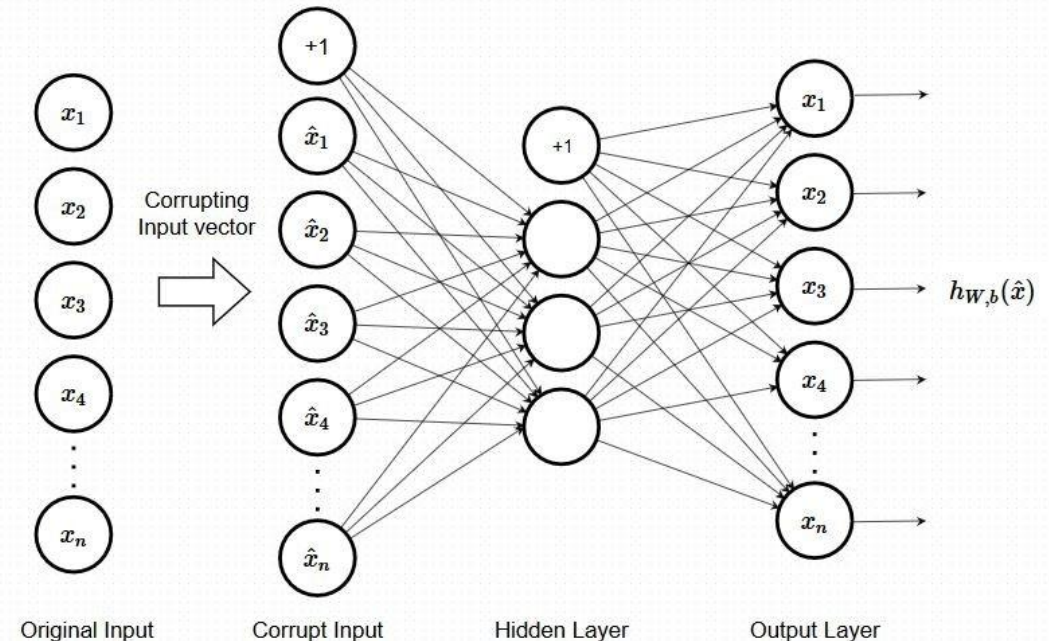


Skip-gram

Modele samonadzorowane

Denoising autoencoder (DAE)

DAE otrzymuje na wejście obraz zaszumiony, następnie uczy się go odszumiać. Nauczony model tworzy w warstwie ukrytej skompresowane reprezentacje danych.

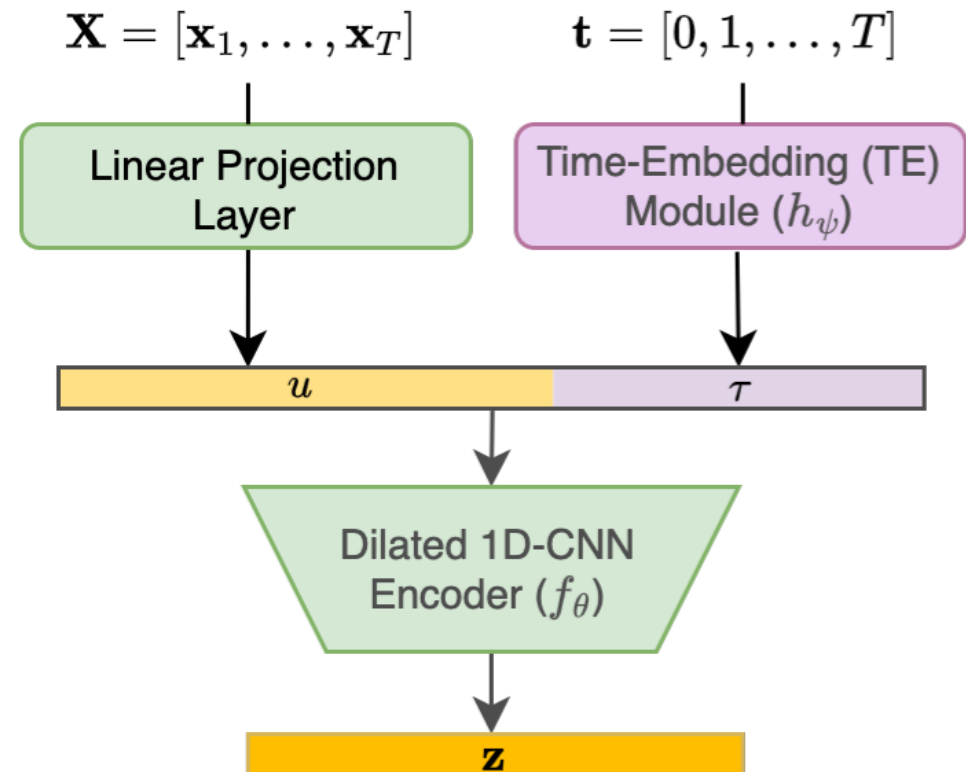


Modele samonadzorowane

T-Rep

Model uczący się reprezentacji szeregów czasowych. Oprócz zwykłego enkodera, transformującego wartości szeregu czasowego, posiada również moduł kodujący samą zmienność czasową.

T-Rep tworzy dodatkowe przykłady uczące, tworząc nachodzące na siebie okna z szeregu czasowego (**nazywane kontekstami**), na które nakładane jest dodatkowo losowe maskowanie.

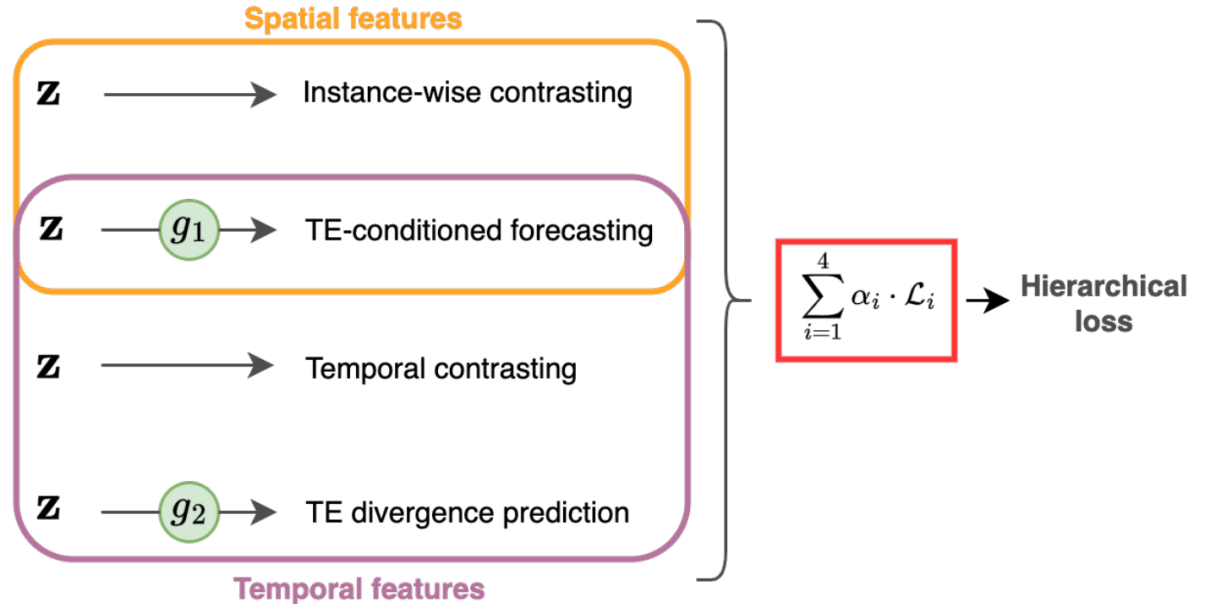


Modele samonadzorowane

T-Rep

T-Rep tworzy reprezentacja w oparciu o kilka zadań-pretekstów:

- **Instance-wise contrasting** - powoduje, że ten sam fragment szeregu w różnych kontekstach będzie reprezentowany podobnie, a różne fragmenty szeregu będą od siebie odpychane.
- **Temporal contrasting** – sprawia, że różne konteksty będą kodowane podobnie, jeżeli ich znaczniki czasowe się zgadzają lub różnie, jeżeli znaczniki się nie zgadzają.



Źródła

1. Książka - LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. „**Deep learning**”
2. **FitNets** - <https://arxiv.org/abs/1412.6550>
3. **Fine-tuning metaanaliza** - Xu, Lingling, et al. "Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment." *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2026).
4. **Adapter tuning** - <https://arxiv.org/abs/1902.00751>; Houlsby, Neil, et al. "Parameter-efficient transfer learning for NLP." *International conference on machine learning*. PMLR, 2019.
5. **LoRA** - <https://arxiv.org/abs/2106.09685>; Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." *Iclr* 1.2 (2022): 3.
6. **Word2vec** - <https://arxiv.org/abs/1301.3781>; Mikolov, Tomas, et al. "Efficient estimation of word representations in vector space." *arXiv preprint arXiv:1301.3781* (2013).
7. **T-Rep** - <https://arxiv.org/abs/2310.04486>

Dziękuję za uwagę!

